The `WebElementState` class provides a large number of methods that tests can use to query and make assertions about the state of a web element. For example, using this class, you could rewrite the test logic like this:

```
bookingPage.searchButton().shouldBeEnabled();
```

If this should fail, the error message will be relatively descriptive:

```
Field '<button id='search'>' should be enabled
```

This way, you can have the best of both worlds: flexible, reusable assertions about the state of your pages that you can place in the test logic, and useful error messages when a test fails.

NAVIGATING WITH PAGE OBJECTS

If an action on a page object causes the application to switch to another page, it's sometimes useful to have the page object return a page object representing the new page. For example, you might write a page object to represent the main menu bar that appears at the top of the page:

```
public class MainMenu {
    ...
    public PlanningPage selectPlanFlight() {...}
    public BookingPage selectBookFlight() {...}
    public FlyPage selectFly() {...}
}
```

You could now write fluent and readable test code like this:

```
mainMenu.selectBookFlight().setFrom("Sydney").setTo("Sydney").search();
```

This approach is often a matter of style. It can make the tests quicker and easier to write, but it may mask the navigation logic through the application and make maintenance harder if the navigation logic changes. It's also debatable whether it's the responsibility of the page object to understand the application's navigation logic, or whether it should be left to the test logic to declare what page it expects to see at any point in the test. Many practitioners prefer to leave the test logic to implicitly describe the expected navigation by using the page object that corresponds to the screen they expect to be on.

If there's a possibility that the action won't always go to another page, or that it may go to multiple pages depending on application-logic considerations, then it's unwise to code this navigation logic into your page objects. For example, a login page will go to a welcome page if the login succeeds or stay on the login page if an authentication error occurs. In this case, the page object will have to decide which page it should navigate to, which is effectively embedding business logic into the page object, and if this business logic changes, your page object will break.

WebDriver is an excellent foundation for automated web tests, and many other open source libraries have been built around WebDriver for more advanced or specific usages. In the next section, we'll look at a few of these.